BeTree — Project Report

Measure where the city is dying, pay people to fix it, prove it worked.

Team **be tree** · HackXplore 2026 · Karlsruhe Citizen app `betree.me` · API `api.betree.me`

---

1. Executive summary

German cities are now **legally required** (Bundes-Klimaanpassungsgesetz) to measure and act on heat and drought — but they have **zero per-tree data**. Karlsruhe alone has tens of thousands of street trees on a ~€1M/year watering budget, watered on fixed truck routes: trees that don't need water get it, trees that do die. A young tree that dies costs **€5,000–15,000** to replace.

BeTree closes two gaps at once:

1. **Real per-tree data** from a cheap solar ESP32 mesh sensor — soil moisture, temperature, footfall and noise from one €24 build.
2. **A verified citizen action loop** — people find a thirsty tree in the app, water it, and the *sensor confirms a real moisture rise* (rain cross-checked). No honor system. Verified care earns credits redeemable for real city perks.

The economics are decisive: **one saved tree pays for a full season of citizen rewards roughly 10–30× over**, and the same sensor answers questions for **five city departments** from one install. The city buys the climate-compliance data it's legally mandated to produce anyway — and citizen labour does the saves.

This is a working product, not a concept

Proof	What exists today
Hardware mesh	ESP32 dual-node **ESP-NOW mesh** firmware (PlatformIO), low-power sensor node + WiFi gateway
Self-built prototype	We 3D-designed & printed the **"Sprig"** enclosure and built it with sensors bought **Saturday morning in a Karlsruhe shop** — it's live in a tree
Live data	**60,000+** real readings from **two live nodes** — one standalone, one mesh (KA-00001 / KA-00002)
Real city	**126,434** actual Karlsruhe trees (open cadastre) on the live map
Real model	FAO-56 soil-water balance + Open-Meteo/DWD weather, reading **root-zone depth**
Shipped	Deployed PWA + FastAPI, green CI (ruff · mypy · pytest, Python 3.11–3.13)

---

2. The problem

- **The mandate exists, the visibility doesn't.** Cities must report and act on climate risk, but manage trees with clipboards and weather-model guesses. Nobody knows which individual tree is dying.
- **Watering is blind and expensive.** Fixed truck routes waste water on healthy trees and miss thirsty ones. Establishment-phase trees (age ≤ 5) die fastest and cost the most to replace.

- **Existing citizen apps run on trust.** "I watered this tree" — self-reported, unverifiable, gameable. A €1M problem running on a paper receipt.

---

3. The solution

A three-sided platform on one shared data layer:

- **Citizens** get a mobile PWA: a live map of every tree's thirst, turn-by-turn to the nearest rescue, QR/NFC tap-to-water, a live moisture chart, confetti on verified success, and a reward shop.
- **The city** gets a dashboard: health overview, footfall and heat heatmaps, watering-route hints, and an exportable audit trail of every verified watering.
- **Five departments** get continuous street-level data (water, heat, footfall, noise, storm tilt) as a *side effect* of the tree network.

---

4. What we built this weekend (proof of realness)

We deliberately built **down the whole stack**, not just a pretty front end:

- **Hardware.** Designed the **Sprig** sensor housing in CAD (`3d/SprigV2.step`), 3D-printed it, and assembled it with a capacitive soil-moisture probe, DHT11, vibration sensor and microphone — **components we bought Saturday morning at a shop in Karlsruhe.** It's staked in a live tree, powered, and reporting.
- **Firmware.** A real **dual-node ESP-NOW mesh**: low-power sensor nodes deep-sleep and relay JSON over ESP-NOW to a WiFi gateway that POSTs to the backend. Each node declares which sensors it has; the backend accepts partial payloads.
- **Backend.** FastAPI + async SQLAlchemy, sensor ingest, tree/forecast/rescue endpoints, gamification, rewards, city dashboard — with **60k+ real readings** in the DB and the **126k-tree** Karlsruhe cadastre on the map.
- **Model.** A FAO-56 soil-water balance coupled to live Open-Meteo/DWD weather.
- **App.** A deployed, installable PWA with the full citizen + city experience.

The Sprig prototype: 3D-printed housing, off-the-shelf sensors, an ESP32, running our mesh firmware.

5. How the technology works

5.1 One cheap node, five data streams

Sensor	Measures	City value
Capacitive soil moisture	Root-zone hydration	*Which trees need water now?*
DHT11 temp + humidity	Per-tree microclimate	Urban heat-island mapping
Vibration / shock	Footfall + storm tilt	Pedestrian counts · storm-damage alerts
Microphone	Ambient sound level	Real-time noise map
Solar + LiPo	Self-powered	Deploy-and-forget

~€24 bill of materials at volume. Nodes mesh over **ESP-NOW** at zero per-node connectivity cost; a single gateway bridges a cluster to WiFi. LoRaWAN (5 km range, existing TTN infrastructure) is a radio swap on the roadmap.

5.2 Seeing into the depths — and covering the city with sparse hardware

A capacitive probe only reads the **top few centimetres** of soil. But a tree drinks from the **root zone, 20–30 cm down** — which can be bone-dry while the surface looks fine. So we don't trust the surface reading alone. We run a **FAO-56 soil-water balance** (the standard agronomic model) driven by free **Open-Meteo / DWD ICON-D2** weather (2 km grid over Germany):

- **Loss** = reference evapotranspiration (ET_0) × crop coefficient (per species) × a water-stress factor
- **Gain** = effective rainfall (canopy throughfall) + known citizen waterings
- **Depth** = the model explicitly pulls Open-Meteo's modelled **deep-soil moisture (3–9 cm, 9–27 cm)** as a physical prior, so it reasons about the root zone, not just the skin of the soil.

The model is **anchored to a real sensor where one exists, and inferred from weather physics everywhere else.** That is the key to scale: **a few hundred calibration sensors ground-truth a physical model that then predicts water stress for all 126,000 trees** — sensored and unsensored alike. Honest about uncertainty: confidence drops with distance from the nearest sensor, and the map can show that. *Sparse hardware, citywide coverage.*

> We present the model as **"physics-based, calibrates as sensors deploy"** — the constants are honest physical estimates, not a trained accuracy number we can't yet back.

5.3 Proof of Care — sensor-verified, tamper-resistant

A watering only earns credit if the **soil agrees**:

- The sensor must show a **sustained moisture rise** during the session — a single spike doesn't count.
- The rise is **cross-checked against live rainfall** — credit is suppressed if it's raining.
- The session has a minimum window — no fake tap-and-run.

The city gets **ground-truth**, not citizen diaries — exactly what the Klimaanpassungsgesetz needs. *(Verification runs in the app today against the live sensor; hardening it server-side is the next step.)*

---

6. One sensor, five departments

The network isn't a tree sensor — it's the **first street-level data grid** for the city. One install, one API, five paying stakeholders:

Department	Question the same sensor answers
Grünflächenamt	Which of our trees need water right now?
Stadtplanung	Did pedestrianising this street change foot traffic?
Umweltamt	Did Tempo-30 reduce noise on residential streets?
Gesundheitsamt	Where are heat islands exceeding 38 °C in a heat wave?
Tiefbauamt	Which trees are leaning after last night's storm?

Each department currently pays for one-off studies and manual surveys to answer these. BeTree gives them **continuous, per-street answers** — and each co-funds a fraction, so the sensor pays for itself before a single tree is saved.

---

7. Costs & economics

7.1 Per-node cost

Item	Approx. cost (at volume)
ESP32 + capacitive moisture + DHT11 + vibration + mic	**~€24 BOM**
Connectivity (ESP-NOW mesh)	**€0 per node** (shared gateway)
Power (solar + LiPo)	self-powered, no grid, ~zero maintenance

7.2 The order-of-magnitude argument

	Cost
Replacing **one** dead young tree (removal + replant + 3–5 yr establishment watering)	**€5,000–15,000**
A **full summer** of citizen watering rewards to keep a tree alive (redeemable perks)	**up to ~€500**
Return on a single saved tree	**~10–30×**

Reward catalogue (low marginal cost to the city): seed packets · museum entry · transit day-pass · priority Bürgeramt appointment. The reward is mostly **existing city capacity**, not cash out the door.

7.3 Pilot budget (illustrative)

A **50-node, one-district, one-summer** pilot: ~€1,200 in hardware + a gateway, the app and API already built, weather data free. If the network keeps **even one or two trees alive that would otherwise die, it has already paid for itself** — before counting the cross-department data value.

---

8. Scalability & realisability

Scales like software, sells like infrastructure.

- **Coverage scales with the model, not the hardware.** Because the physical model generalises from a few hundred calibration nodes to the whole cadastre, you don't need a sensor per tree. Coverage of 126k trees is a software property; accuracy tightens as you add sensors.
- **Connectivity scales cheaply.** ESP-NOW mesh is free per node; LoRaWAN (roadmap) reuses existing TTN gateways for km-scale range with no per-tree WiFi.
- **Data scales the customer base.** One install serves five departments — the addressable value grows without more hardware.
- **The software is production-shaped already.** Async FastAPI, typed schemas, CI on three Python versions, a clean migration path SQLite → Postgres/PostGIS for spatial queries.

Realisability check — what's real vs. roadmap today:

Real now	Roadmap
ESP-NOW mesh firmware, 3D-printed node, live sensor	LoRaWAN radio swap
60k+ readings, 126k-tree map, deployed app + API	Server-side verification hardening (anti-gaming)
FAO-56 + Open-Meteo model, forecast endpoint	Calibrate constants on real sensor history; ML residual layer
CI, tests, typed code	Postgres + PostGIS, push notifications

9. Technical feasibility

- **Frontend:** Next.js 16, React 19, TypeScript, Tailwind 4, MapLibre GL vector tiles, Zustand, PWA.
- **Backend:** FastAPI, async SQLAlchemy 2, SQLite (→ Postgres/PostGIS), Pydantic 2, Open-Meteo client.
- **Firmware:** ESP32 (Arduino/PlatformIO), ESP-NOW mesh, modular per-node sensor config.
- **Quality:** ruff + mypy + pytest in GitHub Actions across Python 3.11/3.12/3.13.
- **Model:** pure, unit-tested water-balance functions; free weather data, cached per neighbourhood.

Every claim above is backed by code in this repository — see the root README.

10. Timeline

Hackathon weekend (HackXplore 2026):

- **Fri** — Concept, data model, FastAPI scaffold, Karlsruhe cadastre ingested (126k trees).
- **Sat AM** — Bought sensors in a Karlsruhe shop; wired the first node.
- **Sat** — 3D-designed & printed the Sprig housing; wrote the ESP-NOW mesh firmware; live readings flowing.
- **Sat–Sun** — FAO-56 + Open-Meteo model; citizen PWA (map, scan/verify, rewards, impact); city dashboard.
- **Sun** — Deployed `betree.me` + `api.betree.me`; green CI; pitch.

Post-hackathon roadmap: server-side verification → calibrate the model on real history → LoRaWAN → Postgres/PostGIS → ML residual correction → multi-department data exports.

11. Risks & honest limitations

- **Model not yet calibrated.** Constants are physics-shaped estimates; we deliberately quote no accuracy figure until we fit on real dry-down data.
- **Verification is app-side today.** Anti-gaming logic exists in the flow but must move server-side to be tamper-proof at scale — it's the first roadmap item.
- **Two live sensors** (one standalone, one mesh). The pipeline is real end-to-end; statistical confidence across the cadastre needs more nodes.
- **Secrets hygiene.** The demo firmware embeds credentials — to be moved to gitignored config and the key rotated before any public release (tracked in `SUBMISSION_CLEANUP.md`).

We'd rather show a jury a small, honest, *working* slice than an impressive number we can't defend.

BeTree — because the trees can't ask for help themselves, but your phone can.